

Projeto Prático #1: O projeto deve ser desenvolvido por grupos de *no máximo* três alunos. É estritamente proibida a partilha de código entre alunos de grupos diferentes.

Plágio: Além de inspeção manual, será também executado um software de deteção de plágio no código-fonte do projeto. *Todos* os alunos que submeterem código obtido através de plágio terão os seus projetos anulados.

Uso de Inteligência Artificial: É permitido utilizar ferramentas de IA como o ChatGPT, Copilot, DeepSeek e similares. No entanto, todos os membros do grupo devem ser capazes de compreender e *explicar* o projeto *na sua totalidade* aos docentes quando pedido. Se houver código no projeto que o grupo não consiga explicar, o projeto será considerado como *plágio*.

Submissão: A data de submissão do projeto é **02/05/2025** às 23:59. A descrição do processo de submissão pode ser encontrada no enunciado.

SoccerNow: Gestão de Jogos de Futsal

1 Contextualização

O clube desportivo **Ciências Soccer Society (CSS¹)** organiza, de forma recorrente, um conjunto de jogos e torneios de futsal. Surge, assim, a necessidade de um sistema que cubra todas as etapas deste processo.

Um utilizador pode interagir com o sistema de duas formas: como jogador ou como árbitro. Os jogadores podem integrar várias equipas e devem ser capazes de indicar a posição que preferem jogar. Por sua vez, o papel de árbitro pode ser desempenhado por um utilizador com ou sem certificado. Jogadores não podem ser árbitros e vice-versa.

Para a gestão de equipas, o sistema permite criar equipas com um nome, manter um histórico dos jogos em que participaram e das conquistas, isto é, posições de pódio em campeonatos. É também necessário registar os jogadores que compõem cada equipa. As equipas não têm limite de tamanho, e cada jogador pode integrar várias equipas.

No que diz respeito aos jogos de futsal, deverá ser possível criar jogos amigáveis, sem associação a qualquer campeonato, bem como jogos que façam parte de um campeonato. A cada jogo associa-se uma data, horário, local, equipas e estatísticas relevantes. Os organizadores devem poder atribuir um ou mais árbitros aos jogos. Se houver mais de um árbitro, um deles deverá ser considerado o principal. Cada jogo tem duas equipas de cinco (5) jogadores, dos quais um será designado como guarda-redes.

A criação e organização de campeonatos envolve a definição de diferentes modalidades. Todos os campeonatos serão disputados por pontos, mas pretende-se estender (no futuro) o sistema para que seja suportada também a funcionalidade de campeonatos no formato de *eliminatória*. Um campeonato é disputado por um mínimo de 8 equipas e é necessária a presença de, pelo menos, um árbitro certificado nas partidas de campeonato.

Por fim, o registo de resultados e estatísticas deve permitir que, no final de cada jogo, se registe o resultado final (placar), a equipa vitoriosa, bem como quaisquer cartões atribuídos a jogadores. Em competições por pontos, o sistema atualiza a pontuação e a classificação das equipas. No formato de eliminatória, define-se quem avança para a fase seguinte.

¹Entidade fictícia

2 Casos de Uso

Numa versão inicial, pretende-se implementar alguns casos de uso, deixando outros para a segunda fase. Apresenta-se, contudo, a lista completa dos casos de uso, de forma a que os alunos tenham uma noção do desenvolvimento que será necessário efetuar na segunda fase. Para a primeira fase, devem implementar os casos de uso identificados com F_1 . Os modelos e diagramas devem ser elaborados considerando todos os casos de uso.

- A. Login com autenticação (Nota: vamos fazer mock e qualquer palavra-passe será aceite).
- B. (F_1) **Registo de utilizadores.**
- C. (F_1) **Verificar, remover e atualizar um utilizador.** Garanta que a lógica da aplicação se mantém consistente.
- D. (F_1) **Buscar por jogadores.**
- E. (F_1) **Criação de equipas.**
- F. (F_1) **Verificar, remover e atualizar as equipas.** A lógica da aplicação deve manter-se consistente (e.g., não é possível remover uma equipa que ainda tenha jogos marcados, ...).
- G. (F_1) **Buscar por equipas.**
- H. (F_1) **Criação de jogos.**
- I. (F_1) **Registar o resultado de um jogo.**
- J. Criação de campeonatos.
- K. Buscar por campeonatos com filtros.
- L. Verificar, remover e atualizar os campeonatos.
- M. Cancelamento de um jogo de campeonato.

Cada aluno deverá implementar um conjunto específico de casos de uso (claramente identificados no repositório). A avaliação do componente de implementação será feita de forma individual, com base no conjunto de casos de uso. Para equipas compostas por dois alunos, é permitido optar por não implementar um dos conjuntos. Nos casos em que haja dependência da implementação do outro integrante, deverão ser utilizados *stubs* até se ter a implementação final. Para a segunda fase do projecto, todos os casos de uso anotados com F_1 devem estar implementados.

Os conjuntos de casos de uso são definidos da seguinte forma:

Conjunto	Casos de uso
C1	B, C e D
C2	E, F e G
C3	H e I

Table 1: Conjuntos de casos de uso

3 Requisitos Não Funcionais

O projecto tem ainda os seguintes requisitos:

- A autenticação poderá ser feita via mock (futura integração com sistema real).
- Toda a informação deverá ser armazenada numa base de dados relacional.
- A plataforma deverá ser implementada em Spring Boot, usando Java. A camada de dados será implementada utilizando JPA/Spring Data.
- A aplicação deverá lidar com pedidos concorrentes, sem criar inconsistências.
- A camada de negócios deverá usar um Domain Model rico e anotações JPA para facilitar o mapeamento.
- A interface será através de *endpoints REST*. Deve ser possível aceder diretamente a esses endpoints através do Swagger (<https://swagger.io/>). Deve-se ter em consideração que haverá uma interface web no futuro.
- O repositório deve aceitar apenas código com o nível de qualidade aceite pela equipa (ex.: *pre-commit*, testes).
- O projecto correrá num ambiente Docker (será a forma de deploy no servidor de produção).
- O controlo da participação de cada membro do grupo será feito via atividade no repositório git, com base no número e qualidade dos *commits*.
- As decisões técnicas devem ser justificadas num relatório técnico.

4 Tarefas da Fase 1

A equipa deverá:

- Fazer fork do repositório original (<https://git.alunos.di.fc.ul.pt/css000/soccernow>), para a conta de um dos elementos do grupo.
- Dar acesso a todos os membros do grupo, como Maintainer.
- Dar acesso à conta css000, como Reporter.
- Alterar o Readme.md para identificar a equipa e indicar qual conjunto de casos de uso cada membro é responsável por implementar.
- Criar uma pasta *docs*, onde se colocará um único documento PDF com todos os diagramas resultantes do projecto.
- Desenhar o modelo de domínio, tendo em conta todos os casos de uso, e colocá-lo na pasta *docs*.
- Desenhar o diagrama de sequência (SSD) para o caso de uso H.
- Desenhar um esboço do diagrama de classes que mostre a divisão em camadas.

- Anotar as classes relevantes com as anotações JPA, de forma a realizar o melhor mapeamento possível.
- Justificar no relatório o porquê de cada decisão tomada neste mapeamento.
- Incluir no relatório as garantias que o sistema oferece relativamente à lógica de negócio.
- Gerar a base de dados a partir das anotações escritas.
- Implementar testes para garantir que a lógica de negócio está correta.
- Implementar os *endpoints REST* necessários (com acesso através do Swagger) para os casos de uso implementados.

Nesta etapa não deverá ser entregue a aplicação completa e também não será necessária interface gráfica. Contudo, devem ter implementados os casos de uso identificados com F_1 na secção 2. Deve-se verificar que a base de dados está correta, de acordo com os princípios aprendidos na disciplina de Bases de Dados e em Construção de Sistemas de Software. Para isso, devem garantir que existem testes e tornar acessíveis os endpoints para a realização dos casos de uso implementados. Todos os casos de uso implementados devem ser funcionalmente corretos.

Para validar o funcionamento da aplicação, é fundamental que o sistema seja capaz de responder a questões pertinentes sobre os dados armazenados. A seguir, apresenta-se um conjunto de exemplos de perguntas que demonstram as funcionalidades esperadas do sistema²:

- Quais as equipas que possuem menos de 5 jogadores?
- Em média, quantos golos os jogadores com o nome “X” marcam por jogo?
- Quais as equipas que recebem mais cartões (amarelos e vermelhos)?
- Quais as equipas com o maior número de vitórias?
- Qual o árbitro que oficiou o maior número de jogos?
- Quais os jogadores que receberam mais cartões vermelhos?

5 Como Entregar

Para entregar o trabalho, basta criar uma tag chamada *fase1* e enviá-la para o repositório. O repositório deverá conter o código-fonte do projecto, os ficheiros necessários para o correr num ambiente Docker, bem como um único documento PDF na pasta *docs* com todos os diagramas necessários.

```
git tag fase1
git push origin fase1
```

Deve-se confirmar que o vosso projecto está acessível à conta CSS000 na tag fase1. Caso contrário, terão 0 nesta entrega. É importante garantir que o ambiente de execução possa ser reproduzido pela equipa docente sem erros de compilação ou outros que impeçam a sua execução. Para tal, recomenda-se executar o projecto em cada um dos computadores dos membros da equipa. O projecto **obrigatoriamente** deverá estar num ambiente Docker. Falhas em **EXECUTAR** o projecto, num ambiente Docker, pela equipa docente, resultarão em **grave penalização nesta entrega**.

²Numa equipa composta por apenas dois alunos, alguns dos casos de uso necessários para responder a todas as perguntas podem não ter sido implementados.

6 Critérios de Avaliação

Esta entrega corresponde a 40% da nota do projecto. Os restantes 60% serão avaliados na segunda fase. Para perceberem onde se devem focar, segue uma lista de critérios que serão avaliados:

6.1 Processo de Desenvolvimento

- Granularidade e frequência dos *commits*. Todos os membros do grupo devem contribuir de forma equitativa. *Disparidades significativas serão penalizadas*.
- Qualidade das mensagens de *commit*.
- Qualidade do código presente no repositório.
- Funcionalidade dos casos de uso.

6.2 Requisitos

- Em que medida os requisitos funcionais e não funcionais são abrangidos pela modelação.
- Implementação da REST API com os endpoints adequados.

6.3 Testes

- Cobertura dos casos de uso pelos testes escritos.
- Cobertura dos casos extraordinários pelos testes escritos.

6.4 Relatório

- Descrição da arquitetura em camadas.
- Qualidade do Modelo de Domínio.
- Qualidade do Diagrama de Classes e do SSD designado.
- Qualidade do mapeamento JPA (incluindo herança).
- Qualidade da justificação das decisões do mapeamento.
- Qualidade da justificação das decisões de negócio.

Nota Final: Em caso de dúvidas, utilizem o fórum de discussão no Moodle. Boa sorte com o desenvolvimento do *SoccerNow*!